Seminar 07 — AI-Assisted Development

PB138 — Basics of Web Development

"Vibe coding is a vibe. Shipping is a discipline."

Agenda

1. **How AI actually works** — tokens, system prompts, attention, hallucinations (~10 min)
2. **The AI coding landscape** — approaches, tools, competitors (~10 min)
3. **Vibe coding vs. agentic coding** (~5 min)
4. **Claude Code deep dive** — CLAUDE.md, skills, hooks, MCP, subagents (~10 min)
5. **Live demo** — building a feature together (~remaining time)

How AI Actually Works

Before you use a tool, understand what it is

What is a Token?

LLMs don't read words — they read **tokens**.

```
"unhelpful"   → ["un", "help", "ful"]
"useState"    → ["use", "State"]
"<button>"    → ["<", "button", ">"]
```

- A token is roughly **3/4 of a word** on average
- ~750 words ≈ 1000 tokens
- Code tends to tokenize **less efficiently** than prose
- Numbers, symbols, and rare words split into more tokens

Why it matters: you pay per token, and the model has a hard limit.

The Context Window = Working Memory

```
┌──────────────────────────────────────────────────────────┐
│          Context Window (e.g. 200K tokens)               │
│                                                          │
│ system prompt │ CLAUDE.md │ conversation + files │▓▓▓▓▓▓│
└──────────────────────────────────────────────────────────┘
                                          you are here ↑
```

- Everything the model can "see" at once lives in the context window
- When context is full, **earlier content is dropped**
- `CLAUDE.md` lives at the top of context — keep it lean, not bloated
- Bigger window = more files, more history, better decisions

System Prompt — Hidden Instructions

Every LLM conversation starts with a **system prompt** — text that the user doesn't see.

```
┌──────────────────────────────────────────┐
│ System prompt (hidden)                   │
│ "You are a helpful coding assistant.     │
│  Always use TypeScript. Follow ESLint..." │
├──────────────────────────────────────────┤
│ User message (visible)                   │
│ "Add a login form"                       │
├──────────────────────────────────────────┤
│ Assistant response                       │
└──────────────────────────────────────────┘
```

- Sets the **persona, rules, and constraints** for the model
- `CLAUDE.md` is injected into the system prompt — that's why it works
- Tools like Claude Code add their own system prompt on top of yours
- The system prompt **counts against** the context window

Context Windows in 2026

Model	Context Window	Notes
GPT-5.4 (OpenAI)	272K / 1M tokens	1M via API with 2x pricing
Claude Sonnet 4.6	200K tokens	Anthropic's fast model
Claude Opus 4.6	200K / 1M tokens	Anthropic's most capable
Gemini 3.1 Pro	1M tokens	Google, native multimodal
Llama 4 Scout (Meta)	10M tokens	Open source, MoE architecture

The trend: context windows keep growing. More context = better results.

Attention — How the Model "Reads" Code

Every token **attends to every other token** — the model connects distant references.

```
function calculateTotal(items) {  // ← defined here
  return items.reduce(...)
}

// 300 lines later...
const result = calculateTotal(cart) // ← model connects these
```

- Tracks **long-range dependencies** across the whole context
- This is why pasting full files gives better results than snippets
- More context = better understanding, but with diminishing returns

Why AI Hallucinates

LLMs predict the **most probable next token** — they don't "know" facts.

```
// Model generates this confidently:
import { useSomething } from "@tanstack/react-query";
//       ^^^^^^^^^^^^^^ doesn't exist! But it looks plausible.
```

- No strong signal in training data → **confident wrong answer**
- The model has never "run" code — it pattern-matches
- API names, library versions, function signatures = common failures

"The model is always trying to be helpful. That's the problem."

Defense: Always verify. Run the code. Use **Context7 MCP** to give AI live docs.

Prompts Shape Output

Vague prompt → vague output. Context-rich prompt → precise output.

Prompt	Result
`"add a button"`	Generic, guesses everything
`"add a submit button with Shadcn variant='default'"`	Correct, uses project patterns
`"add a submit button"` + lean CLAUDE.md	Correct without spelling it out

The better your context, the less you have to say.

Prompting Techniques

- **Chain-of-thought** — "think step by step" improves complex logic
- **Few-shot examples** — show the format you want, the model follows it
- **Constraints** — "use only standard library" prevents hallucinated deps
- **Specificity** — file names, frameworks, patterns > vague descriptions

The AI Coding Landscape

Know what you're choosing and why

The Spectrum

```
Manual coding ──────────────────────────────── Fully autonomous
     │                                                         │
     │  Code         Chat      Spec-driven   Vibe              │
     │  Completion  Assistants  Agentic     Coding             │
     │                                                         │
     │  You write   You ask   AI implements AI does            │
     │  AI suggests AI answers You review   everything
```

Each approach trades **control** for **speed**. Most developers use 2-3 of these depending on the task.

1. Code Completion

What: AI suggests the next few lines as you type.

Tools: GitHub Copilot, Supermaven, JetBrains AI

```
// You type:
function calculateTotal(items: CartItem[]) {
  // Copilot suggests:
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}
```

Pros: Fast, non-intrusive, stays in your flow

Cons: Limited context, line-by-line, no big-picture awareness

2. Chat Assistants

What: You describe what you need, AI generates code in a chat.

Tools: ChatGPT, Claude.ai, Gemini

```
You: "Write a React hook that debounces a value by 300ms"
AI:  *generates full hook with types and explanation*
```

Pros: Great for learning, exploration, one-off snippets

Cons: Copy-paste workflow, no project context, no file access

3. Agentic Coding

What: AI operates directly in your codebase — reads files, writes code, runs commands.

Tools: Claude Code, Cursor, Windsurf, Antigravity, Codex, Kiro

```
$ claude
> Add a dark mode toggle to the header

# Claude reads your codebase, installs packages,
# edits files, runs tests — you review the diff
```

Pros: Full project context, multi-file edits, runs tests

Cons: Can make mistakes — review needed if you care about code quality, costs tokens

AI Coding Tools in 2026

Tool	Type	Notable Feature
GitHub Copilot	Completion + Agent	$10/mo, 300 premium reqs, agent mode
Cursor	Full IDE (VS Code fork)	1M+ users, deep AI integration
Windsurf	Full IDE (VS Code fork)	Acquired by Cognition (Devin)
Antigravity	Agent-first IDE (Google)	Free, built-in browser, multi-agent
Kiro	Spec-first IDE (Amazon)	Auto-generates specs before code
Codex	Cloud agent (OpenAI)	Sandboxed, pushes to GitHub
Claude Code	CLI agent (Anthropic)	Skills, hooks, MCP, subagents
Gemini CLI	CLI agent (Google)	1M context, free tier

Vibe Coding vs. Agentic Coding

One is a feeling. The other is a practice.

What is Vibe Coding?

A term coined by **Andrej Karpathy** in 2025.

```
You: "make it look nicer"
AI:  [generates 200 lines]
You: "looks good, ship it"
```

- Prompt → accept → ship, with **minimal understanding**
- You don't read the code. You **feel** whether it works.
- For prototypes and throwaway projects: **totally fine**.

The Vibe Coding Trap

The problem isn't the first feature. It's the fifth.

```
Week 1:  app works, vibes are immaculate ✓
Week 3:  "why is there a useEffect here?"
Week 5:  adding a button breaks auth somehow
Week 8:  nobody knows how anything works
```

- No mental model → bugs are mysterious
- Technical debt accumulates **invisibly**
- The AI is the only one who "understands" it — and it **forgets every session**
- **59 %** developers admit to using AI code they don't fully understand

Vibe coding doesn't scale. It compounds.

Agentic Coding

You are still the engineer. The AI is the **power tool**.

```
You: [write spec]
     "implement user notifications:
      - in-app notifications for new comments
      - mark as read individually or all at once
      - persist state in the database"

AI:  [implements across 6 files, writes tests]
You: [review, evaluate, accept or reject]
```

- You define **what** and **why** — AI handles **how** and **where**
- You **evaluate every output** against your mental model
- You own the spec → you own the outcome

Fun fact: ~90 % of Claude Code's own codebase was written by Claude Code itself.

Claude Code Deep Dive

The toolkit

CLAUDE.md — Your AI's Manual

The file Claude reads **before doing anything** in your project.

```
# Project: Trollo
## Stack
- TanStack Start, Drizzle ORM, SQLite, Tailwind v4, Shadcn/ui
## Patterns
- File-based routing under `src/routes/`
- Server functions co-located with routes
- Use `@/` alias for imports from `src/`
## Commands
- `bun dev` — start dev server
- `bun run check` — lint + format
- `bun db:push` — sync database schema
```

Your primary way to give Claude **persistent context**.

But keep it lean — research shows that bloated context files **reduce** success rates and increase cost by 20 %+. Write only what's universal for every task. AI tends to add rules reactively — prune regularly.

MCP — Model Context Protocol

An open protocol for connecting LLMs to **external tools and data**.

```
Claude Code  ←→  MCP Server  ←→  External System
                 (adapter)
```

```
// .mcp.json — add to project root
{
  "mcpServers": {
    "playwright": { "command": "npx", "args": ["@playwright/mcp@latest"] }
  }
}
```

Examples: Context7 (live docs), Playwright (browser), GitHub (PRs/issues), Figma (designs)

MCP turns Claude from "text in, text out" into an agent that can **interact with the world**.

Skills — Reusable Prompt Workflows

Skills are **version-controlled, shareable** prompts for common tasks.

```
# Install a plugin with skills
/plugin install superpowers@claude-plugins-official

# Use a skill
> /commit            — writes a conventional commit message
> /feature-dev       — guided feature development workflow
> /tdd               — test-driven development loop
```

Each skill is a markdown file that encodes **best practices** into repeatable workflows.

Skills enforce **discipline** — they prevent the AI from jumping to code before understanding the problem.

Hooks — Automation at Every Step

Hooks run shell commands **before or after** Claude's tool calls.

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "bunx biome format --write \"$CLAUDE_TOOL_INPUT_FILE_PATH\""
      }]
    }]
  }
}
```

- Auto-format after every file change
- Run tests before accepting edits
- Log all AI actions for audit trails

Permissions — Safe but Not Annoying

Three levels of settings files, applied in order:

```
~/.claude/settings.json            ← user-level (all projects)
.claude/settings.json              ← project-level (committed, shared)
.claude/settings.local.json        ← local overrides (gitignored)
```

```
{
  "permissions": {
    "allow": ["Edit", "Read", "Bash(bun *)"],
    "deny": ["Bash(git push --force*)"]
  }
}
```

Goal: Claude doesn't interrupt you for safe ops, but asks for anything risky.

Subagents — Divide and Conquer

Claude can spawn **subagents** to work on independent tasks in parallel.

```
Main agent: "implement the Kanban board"
│
├── Subagent A:  update database schema
├── Subagent B:  write server functions
└── Subagent C:  build the UI components
```

- Each subagent has its own **isolated context**
- They work independently, then report back
- The main agent **integrates** the results
- Massively reduces time on large features

Spec-Driven Development

Write the spec first. Let the agent implement it.

```
1. Brainstorming     → Explore requirements, write spec
2. Writing Plans     → Break spec into implementation steps
3. Implementation    → Execute plan step by step (with subagents)
4. Code Review       → Verify quality and correctness
```

This is the workflow we used to build the Kanban board in Trollo. Each step produces an **artifact** (spec, plan, code, review).

"Every good AI setup is secretly a well-documented project."

AI in Numbers (2024–2025)

Stat	Source
62 % developers use AI tools (up from 44 % in 2023)	Stack Overflow 2024
55 % faster task completion with Copilot	GitHub Research
~90 % of Claude Code's own codebase written by Claude Code	Anthropic
82 % of AI users use it primarily for writing code	Stack Overflow 2024
45 % rate AI as "bad" at complex tasks	Stack Overflow 2024
43 % trust AI output; **31 %** are skeptical	Stack Overflow 2024

The tools are good enough to be useful, but not good enough to be trusted blindly.

Slopsquatting — When Hallucinations Become Attacks

AI models hallucinate **fake package names**. Attackers register them with malware.

```
# AI suggests:
npm install react-helper-utils
# ^^^ doesn't exist — but an attacker just registered it
```

Defense: Always verify dependencies exist before installing.

Slopsquatting — Numbers

Fact	Number
Overall hallucination rate for packages	**19.7 %**
Commercial models (GPT-4, Claude)	**~5 %**
Open source models (CodeLlama)	**>33 %**
Hallucinated names that repeat across runs	**58 %**

Source: UT San Antonio et al. — 576K code samples, 16 models, 205K unique fake packages

Claude Code Workflow

Plan first — always ask for a step-by-step plan before coding

Diff over rewrite — review changes as diffs, not full files

Tight loop — generate → review issues → fix → **commit**

Use CLAUDE.md as guardrails — only strict, enforceable rules (keep it minimal)

Leverage tooling — rely on TypeScript, tests, linters as feedback loops

Add constraints — prevent overengineering ("don't add deps, follow patterns")

Prefer minimal changes — avoid "smart" rewrites unless needed

Use AI for review — bugs, edge cases, production risks

When NOT to Trust AI (1/2)

Security-critical code — AI still frequently generates insecure patterns (auth, crypto, validation)
[OWASP Top 10 for LLM Apps, 2024]

Dependencies & supply chain — AI suggests non-existent or malicious packages (*slopsquatting*) *[Kusari Report, 2026]*

Hallucinations & correctness — confident but wrong outputs remain a top issue *[OpenSSF, 2025]*

Complex domain logic — lacks business context → higher risk of logic errors *["Vibe Coding" discussion, 2025]*

When NOT to Trust AI (2/2)

Performance-critical paths — optimizes for “works”, not efficiency or scale *[Chen et al., 2025]*

Architecture decisions — copies patterns, ignores trade-offs and constraints *[Krishnamurthy et al., 2025]*

Long-term code quality — measurable increase in defects and technical debt *[Peng et al., 2023; follow-up studies 2025]*

The 70/30 rule
AI gets you 70% fast.
The last 30% — **correctness, security, edge cases** — is where engineers matter most.

> "AI-augmented software engineering, not automated software engineering." — Addy Osmani

Live Demo

Let's build a feature in Trollo together

Trollo — Our Demo Project

A Kanban board built with the full stack:

Layer	Technology
Framework	TanStack Start
Styling	Tailwind CSS v4
UI Components	Shadcn/ui
Data Fetching	TanStack Query
ORM	Drizzle ORM

Current state: Three columns (To Do, In Progress, Done), drag & drop, CRUD.

Now: Let's add a new feature together using Claude Code.

Let's Go!

```
cd trollo
claude
```